AllSpeak: A Language-Agnostic Runtime for Computational Literacy in Multilingual Communities

Working Paper — May 2026

Author: Graham Trott (independent developer)

Correspondence: info@allspeak.ai

Project repository: https://github.com/easycoder/allspeak.ai

Cite as: Graham Trott (2026). *AllSpeak: A Language-Agnostic Runtime for Computational Literacy in Multilingual Communities*. Zenodo. https://doi.org/10.5281/zenodo.22018537

Abstract

Language is the foundation of human development. It is what enables us to convey ideas, share knowledge, and build on the discoveries of those who came before us. Since the industrial age, we have extended language into new forms—programming languages that instruct and control the machines we build to serve us. Software engineering, expressed in these languages, now underpins the whole of modern society.

We are now creating new forms of intelligence, expected before long to transform every part of our existence. If humans are to retain a relevant role, we must learn to interact usefully with these entities. The traditional programming languages cannot serve this purpose alone: they exclude most of the world's population from the process, because they are expressed in English and designed for human authors, not human readers.

AllSpeak is an open-source runtime designed to be a lingua franca—a common language comprehensible to both humans and AI, available in any human tongue. Its programs are intentionally simple, constrained, and readable, not because the problems they address are trivial, but because legibility is the design goal that makes human oversight possible. A codebase that no human can meaningfully inspect is not a managed system; it is an autonomous one, operating on trust. In high-stakes domains—automation, finance, health, infrastructure—that is a governance problem as much as a technical one.

This paper describes AllSpeak's architecture, its current implementation in four languages, and the methodology developed for extending it to new linguistic communities. It argues that as AI-generated code becomes ubiquitous, the humans who need to understand it will be a different population from those who once wrote it—less technically trained, more linguistically diverse—and that the tools available to them should reflect that reality.

Keywords: computational literacy, multilingual programming, AI-assisted development, digital inclusion, language-agnostic runtime, code comprehension, open source

---

1. Introduction

The global expansion of coding education over the past two decades has been built on a single unstated assumption: that the people learning to code will be the people writing it. Tools, curricula, and communities have been designed accordingly—for the authorship of code, in programming languages that are structurally rooted in English.

That assumption is now being tested. AI systems capable of generating functional code from natural language descriptions are in widespread professional use. GitHub Copilot, launched in 2022, reported over 1.8 million paid subscribers by February 2025 and is integrated into the daily workflow of a substantial proportion of professional developers (GitHub, 2025). The question of who writes code, and whether that skill remains the primary measure of computational literacy, is open in a way it was not five years ago.

AllSpeak is an open-source scripting language designed for a different assumption: that the more important skill, going forward, is the ability to read, understand, and meaningfully engage with code—and that this skill should be available to people regardless of their technical background or native language.

This paper describes AllSpeak's architecture, its current implementation in four languages, and the methodology developed for extending it to new linguistic communities. It also addresses directly the question that any honest treatment must confront: in an era of AI-generated code, what is the continuing value of a human-readable coding layer?

A precise definition

> *"AllSpeak is a linguistically inclusive computational literacy layer—sitting between human understanding and executable logic, regardless of whether a human or an AI generated the underlying code."*

This framing locates AllSpeak in the technology stack without making claims about who is above or below it. It is not a competitor to Python or JavaScript; it occupies a different functional space—one where human comprehension of what the code does is a design requirement, not an afterthought.

Vocabulary over syntax: the "laser" principle

There is a recurring reaction when AllSpeak is shown to professional programmers: doubt that something this simple could possibly handle real-world complexity. It looks like a toy language for toy projects. The doubt is understandable, and it misses the point.

Consider the word "laser." Almost everyone has a working understanding of what a laser does—it produces a concentrated beam of light. Almost no one understands the quantum physics of stimulated emission that makes it work. The word "laser" is a vocabulary item that hides a universe of complexity behind a simple, intuitive label. We don't need to understand the physics to use the tool effectively.

AllSpeak applies the same principle to code. Its vocabulary items—`put`, `create`, `append`, `rest get`, `on click`—are the "lasers" of the system. They hide the underlying complexity of HTTP requests, DOM manipulation, and data flow behind words that anyone can read and understand. The engine beneath them can be arbitrarily sophisticated—and it is, with 127 opcodes, concurrency via fork/wait/every, a full JSON processing pipeline, and plugin extensibility for graphics, maps, MQTT, and more.

A tool is not limited by the simplicity of its vocabulary. It is limited by the breadth of what that

vocabulary can express. AllSpeak's vocabulary covers the full range of application logic—flow control, data manipulation, event handling, network communication, storage, concurrency—in a form so readable that a non-programmer can follow it. That is not a limitation. That is the entire point.

This principle is central to AllSpeak's design and should be understood from the outset: the project is not about making programming languages simpler because the problems they solve are trivial. It is about making the expression of logic accessible to human readers, regardless of the complexity of the underlying machinery.

---

2. The Shifting Role of Code Literacy

2.1 What is already changing

The signals are not predictions—they are present-tense observations. GitHub Copilot, the most widely deployed AI coding assistant, was estimated to be responsible for approximately 30% of new code written on the GitHub platform by mid-2025 (GitHub, 2025). No-code and low-code platforms have expanded the population of people deploying functional software without writing conventional code, with the global market projected to reach $187 billion by 2030 (Forrester Research, 2024). "Prompt engineering" has emerged as a recognised professional role, listed on major job platforms and commanding salaries comparable to junior developers in some markets.

Within the AllSpeak project's own observations, the pattern is visible in the declining attendance at traditional coding meetups and the shift towards AI-assisted development tools among professionals. A series of "learn to code" meetups has seen steady attendance decline over the past year; when the same organiser rebranded an event as an "AI mini unconference," attendance surged. Professional coders who honed their skills over decades report those skills atrophying through disuse—not because they can no longer understand code, but because they are unaccustomed to writing it.

This does not mean people are learning less. A study of a large university coding course found that when AI chat tools were introduced, traditional engagement metrics—homework completion, section attendance—dropped significantly, yet exam performance for those who adopted the tools actually improved (Stanford HAI, 2025). The students were making a calculated choice to learn more efficiently, not opting out of learning altogether.

These trends do not represent the end of programming. They represent a shift in where human skill and attention is applied in the development process. The practical question is what follows from that shift.

2.2 The handwriting parallel

An analogy is offered here not as a prediction but as a frame for thinking. When mechanical text reproduction became widely available—the typewriter, then the word processor—the teaching and practice of handwriting changed fundamentally. The ability to read handwritten text did not disappear; the ability to produce it fluently and elegantly largely did, because it was no longer the primary medium of written expression and was no longer systematically taught.

Whether code authorship follows a similar trajectory is a question reasonable people can disagree on. What is harder to dispute is that the population actively practising code writing is likely to shrink relative to the population that needs to engage with code in some way—reading it, questioning it, modifying it, deciding whether to trust it.

The handwriting analogy has a limit worth naming: handwriting loss is largely benign, whereas code comprehension loss has systemic consequences. A generation that could not produce beautiful handwriting got along fine. A generation that cannot meaningfully inspect the code running their heating systems, financial transactions, and medical devices faces a different kind of vulnerability.

2.3 Reading versus writing

Whatever one believes about the future of code authorship, the need to read and reason about code is not diminishing. The volume of code requiring inspection, validation, and maintenance is increasing. The population capable of performing that inspection is not growing proportionally, and the tools available to support it have not kept pace with the tools available for generating code.

This is a present-tense problem, not a future one. The gap between the rate of code generation and the rate of meaningful human review is already observable in professional contexts. It is likely to widen.

Notably, the same "GPT Surprise" finding—students who used AI tools attended fewer classes but performed better on exams (Stanford HAI, 2025)—reinforces this argument from a different angle. The students were not rejecting the material; they were rejecting the traditional *production-oriented* path through it. They learned the same concepts, tested against the same outcomes, but via a route that minimised conventional code-writing. This is precisely the shift this paper describes: the skill that matters is comprehension of the result, not facility with the process that produces it.

2.4 The closed loop problem

A system in which code is generated, deployed, and operated without meaningful human inspection is, in systems terms, an open loop: functional until it fails, with no internal mechanism for course correction short of that failure. In low-stakes contexts—generating a form letter, producing a data summary—this may be acceptable. In domains where the consequences of error are significant—home automation, financial calculation, medical scheduling, infrastructure control—it is a governance problem.

Constrained, legible runtimes are one class of response to this problem. A language with a small, consistent vocabulary and no syntactic ambiguity is not only easier for humans to read; AI systems themselves make fewer mistakes when generating code in constrained grammars. The simplicity is a feature for both the producer and the reviewer.

AllSpeak is designed for exactly this space. It does not replace general-purpose languages; it occupies the layer where human comprehension is the priority.

It is worth emphasising that this is not a competition. AllSpeak is implemented *in* JavaScript and Python—it depends on them for its very existence. What AllSpeak does is package complex functionality as vocabulary items, so that domain-specific logic can be expressed without requiring the reader to understand the underlying implementation. The general-purpose languages handle the machinery; AllSpeak handles the legibility. Both have to exist, because both serve different parts of the same problem: one enables the complexity to be built, the other enables it to be understood.

2.5 The language dimension

If code literacy—reading, not writing—is the relevant skill for an expanding population, the language in which code is expressed becomes newly important. The population that needs to inspect and reason about AI-generated programs is not primarily composed of English-speaking developers. It includes teachers, community organisers, local administrators, and householders in communities where English is a second or third language.

Presenting code to these readers in English is not a neutral choice. It is a barrier. There are roughly

1.5 billion English speakers in the world and roughly 6.5 billion who are not (Ethnologue, 2025). Every mainstream programming language uses English keywords. AllSpeak addresses this at the syntactic level: the code itself is in the reader's language, not merely the documentation around it.

---

3. Architecture and Approach

3.1 The opcode model

AllSpeak is built around **127 canonical opcodes**—the fixed, language-independent instruction set of the runtime. These opcodes cover the full range of operations: flow control (`WHILE`, `IF`, `GOSUB`), data manipulation (`PUT`, `APPEND`, `SET_PROPERTY`), DOM interaction (`CREATE_ELEMENT`, `SET_STYLE`, `ON_CLICK`), arithmetic (`ADD`, `MULTIPLY`, `DIVIDE`), JSON handling (`JSON_PARSE`, `JSON_SET_LIST`, `JSON_SORT`), event handling (`ON_CLICK`, `ON_CHANGE`, `ON_SWIPE`), concurrency (`FORK`, `WAIT`, `EVERY`), and utility operations (`LOG`, `ALERT`, `FILTER`, `SORT`, `INDEX`).

Each opcode maps to a surface keyword in each language pack. A program written in Italian and a program written in German compile to identical opcode sequences; the execution engine sees only opcodes, never the source language.

3.2 The language pack architecture

Language packs are JavaScript files that export a structured object with the following components:

Component	Purpose	Size (EN reference)
`meta`	Language identifier, label, version	3 fields
`opcodes`	127 opcode definitions, each with keyword and grammar patterns	~970 lines
`connectors`	Connective words (to, into, from, with, by, of, in, as, on, and, or, giving, the)	13 entries
`literals`	Boolean and string-type names (true, false, body, array, object, storage, parent, sender, ready, nowait)	10 entries
`timeUnits`	Time units for wait/every (second, minute, tick)	6 entries
`conditions`	Comparison and type-check words (is, not, greater, less, than, includes, starts, ends, empty, numeric, even, odd)	12 entries
`diagnostics`	Translated error message templates	7 templates
`words`	Bidirectional vocabulary map (~360 entries) connecting every surface word to its canonical form	~360 entries

The `words` section is the heart of the system. Every canonical term in the engine has one or more surface forms in the language; the engine resolves user-facing tokens through a reverse lookup. This is what enables a French user to write `mets` instead of `put`, and an Italian user to write `variabile` instead of `variable`—both map to the same internal opcode.

3.3 Compilation through the language layer

The compilation pipeline works as follows:

1. **Script parsing:** The source text is tokenized by the compiler (`Compile.js`).
2. **Language directive detection:** If the script begins with `language fr` (or `language it`,

`language de`), the compiler loads the corresponding language pack (`AllSpeak_LanguagePack_fr`) and initialises the language layer (`AllSpeak_Language.init(pack)`).

3. **Keyword dispatch:** Each token is looked up in the active language pack's `words` map to determine its canonical form. The compiler then dispatches to the appropriate opcode handler (defined in `Core.js` for core operations, `Browser.js` for DOM operations, `REST.js` for HTTP requests, `MQTT.js` for messaging).
4. **Token comparison:** Throughout compilation, keywords such as `giving`, `into`, `end`, `then`, `or` are resolved through `AllSpeak_Language.word('giving')`, which returns the active language's surface form. There are approximately 50 such call sites in `Core.js` alone, plus more in `Compile.js`, `Browser.js`, `JSON.js`, `MQTT.js`, and `REST.js`.
5. **Bytecode generation:** Each parsed statement is compiled into a command object stamped with its domain and keyword, then resolved through `AllSpeak_Opcodes.resolve()` into the canonical opcode string. The execution engine (`Run.js`) processes these opcodes without reference to the original language.

3.4 The JS/JSON split

Language packs are implemented as JavaScript for structural and behavioural logic. A companion tool (`sync-language-packs`) extracts the JSON vocabulary data from each JS pack and writes it to JSON files for the Python runtime, ensuring parity across implementations. This separation makes community contribution tractable: a translator working on a new language pack can contribute vocabulary and pattern data without engaging with the engine internals.

3.5 The Weblate integration

Community translation and validation is managed through Weblate, hosted at `translate.codeberg.org`. This provides a structured workflow for native-speaker review, version control for language pack changes, and an audit trail for translation decisions. Each language pack is managed as a separate component, with proposed changes tracked through the Weblate review workflow before being merged into the main repository.

---

4. Current Implementation

AllSpeak is currently operational in **four languages**: English (the reference implementation), French, German, and Italian. All four implementations share a single execution engine; they differ only in their language packs.

The three non-English implementations are **functional**—programs can be written and executed correctly—but have not yet undergone formal linguistic validation by native-speaker educators.

This distinction matters. Functional translation means the opcodes map correctly and programs execute as expected. Validated translation means a native speaker with relevant educational experience has confirmed that the vocabulary choices are natural, unambiguous, and appropriate for the target audience. The validation step is a prerequisite for any serious claim of community suitability.

4.1 Validation methodology (proposed)

Validation of a language pack consists of four steps, designed to be carried out by a native speaker with basic programming familiarity:

1. **Vocabulary review:** Each of the 127 opcode keywords, 13 connectors, and ~360 word-map entries is reviewed for naturalness, consistency, and absence of ambiguity in the target language.
2. **Codex walkthrough:** The validator executes the full Codex tutorial sequence (20 steps) in the target language, verifying that every script runs correctly and every explanation reads naturally.
3. **AI collaboration test:** The validator uses an AI coding assistant to generate AllSpeak programs in the target language, verifying that the AI's output is syntactically correct and uses natural vocabulary.
4. **Educator assessment:** A classroom educator or curriculum designer reviews the materials for age-appropriateness, pedagogical suitability, and cultural context.

The validation timeline is estimated at 2–4 weeks per language, depending on the availability of the validator.

4.2 Known i18n gaps

Several minor internationalisation gaps have been identified and documented. None are compile-or-crash blockers for the existing four languages:

- **Uninitialised variable display:** The runtime displays the English word `undefined` when an unassigned variable is interpolated into a string. This should ideally display the localised equivalent (e.g., `non défini`, `nicht definiert`, `non definito`).
- **Compiler diagnostic strings:** Some error messages bypass the language pack's diagnostics system and emit English text (`Compile error in '...'`, `Warnings:`, `Unrecognised syntax in '...'`). These are known and tracked for resolution.
- **Starter-zip UI strings:** The per-language starter zips bundle three shared files (`server.as`, `asedit.as`, `asedit.json`) whose user-visible strings are currently English across all language packs. This is non-blocking because the agent-facing `AGENTS.md` guide—which is what instructs the AI coding assistant—is fully translated.

---

5. The Multilingual Challenge

5.1 Beyond vocabulary

Translating a programming language is not the same as translating a document. Vocabulary is the least of it. The deeper challenges arise from structural differences between languages:

- **Grammatical gender:** In French, German, and Italian, keywords that function as adjectives or participles must agree in gender with their nouns. The language pack must handle multiple forms where English uses a single form.
- **Verb conjugation:** Where English uses the imperative form consistently (`put`, `take`, `set`), other languages may require different conjugations depending on the formality or grammatical construction.
- **Word order:** Sentence-final constructions in German (where the verb moves to the end in subordinate clauses) vs. sentence-medial in English and French. The language pack patterns describe these structural variations.

- **Script:** The most fundamental challenge. Languages using non-Latin scripts (Cyrillic, Arabic, Devanagari, Chinese characters) require the runtime to handle extended character sets, right-to-left text, and potentially collation issues.

AllSpeak's language pack architecture handles these through the `patterns` field in each opcode definition, which describes the accepted grammatical structure for that language, and the `words` map, which handles vocabulary variants including accented forms.

5.2 Non-Latin scripts: roadmap toward a proof point

The inclusion of a non-Latin-script language in AllSpeak is a recognised priority, not yet implemented. Bulgarian (Cyrillic script) has been identified as a strong candidate for this proof point for several reasons:

- **Alphabetic script:** As a Cyrillic alphabet language, Bulgarian represents a step-change from Latin scripts without introducing the additional complexity of logographic or abjad writing systems (which would be the next phase after Cyrillic).
- **EU member state:** Bulgarian is an official language of the European Union, providing a clear use case for multilingual education and administration within an existing policy framework.

The key architectural question for Cyrillic (and for all non-Latin scripts) is whether to: 1. Support the native script directly in the runtime (allowing keywords like `постави`, `вземи`, `ако` for put, take, if), 2. Use Latin transliteration (allowing Bulgarian speakers to type their language in Latin characters), or 3. Offer both options.

The preferred approach is option (1) for the language pack itself, with transliteration available as a fallback for keyboard environments where Cyrillic input is impractical. The runtime already handles Unicode throughout, and existing language packs include accented characters (French `é`, German `ß`, Italian `à`), so the character-encoding infrastructure is in place. What remains is implementation work: creating the Bulgarian language pack, testing it against the Codex, and documenting the methodology for future non-Latin languages including Arabic, Devanagari, Chinese, and others.

5.3 Lessons from existing translations

The three existing non-English translations have already surfaced important patterns:

- **French** required handling the joiner word `que` in multi-word constructs like `tant que X` (equivalent to `while X` in English), solved by adding an optional `skipWord('that')` between the keyword and the condition in the `While.compile` handler.
- **German** exposed a wrong-canonical mismatch: the `attach` command's pattern declared the connective word `an`, but the internal handler expected the canonical `to` form. Fixed by extending the German word map to support `"to": "zu|an"`.
- **Italian** demonstrated that a single knowledgeable person with AI assistance can produce a complete language pack in approximately one day, reinforcing the viability of the methodology described in Section 6.

---

6. Methodology for Language Onboarding

The following describes a replicable process for extending AllSpeak to a new language. It is designed to be executable by a small team—potentially a single motivated individual with AI assistance and

access to native-speaker review.

6.1 AI-assisted initial translation

The reference English implementation provides the source material. An AI language model is used to produce an initial draft translation of the 127 opcode keywords, 13 connectors, 12 conditions, 10 literals, 6 time units, and vocabulary for the word map. The AI is instructed to produce natural, idiomatic keywords appropriate for a programming context in the target language.

This draft is explicitly a starting point, not a finished product. AI translation of technical vocabulary into natural-sounding keywords requires human review—and the quality of AI translation varies by language, being generally weaker for lower-resource languages. The methodology depends on human review precisely for this reason.

6.2 Community review via Weblate

The draft translation is submitted to the project's Weblate instance at `translate.codeberg.org` for community review. Contributors with relevant language expertise can propose alternatives, flag unnatural choices, and discuss edge cases. The Weblate interface provides: - Per-component translation proposals (opcodes, connectors, diagnostics, etc.) - Version-controlled history of all changes - Comment threads for discussion of specific translation choices - Quality checks for consistency across the language pack

6.3 Testing against the Codex

Before any language pack is accepted for distribution, it must pass the Codex validation suite. The AllSpeak Codex—a 20-step tutorial introducing the language incrementally—serves a dual purpose. For learners, it is an on-ramp. For validators, it is a test instrument: a language pack that cannot support a fluent Codex walkthrough is not ready for community use.

The Codex steps are:

#	Step	#	Step
1	Hello World	11	Interactive applications
2	Basic arithmetic	12	Game development (TicTacToe)
3	String handling	13	List manipulation
4	DOM introduction	14	Advanced sorting
5	Styling and CSS	15	List filtering
6	Image handling	16	Geospatial (Google Maps)
7	Simple animation	17	Drag and drop
8	Physics simulation	18	Card game (Solitaire)
9	Visual effects	19	Pan and zoom
10	Debugging tools	20	Image transitions (Ken Burns effect)

Each step exists as a complete, runnable `.as` script across all four existing languages, plus accompanying tutorial text in `codex/<lang>/md/`.

6.4 Educator validation

The reviewed and tested translation is then assessed by at least one native-speaker educator with experience in the target community. Validation criteria include: - Naturalness of vocabulary choices

- Absence of ambiguity in technical contexts - Suitability for the intended age range and educational setting - Appropriate register (formal enough for learning materials, natural enough for everyday use) - Successful execution of all Codex tutorial scripts

6.5 Publication and maintenance

Once validated, the language pack is: 1. Merged into the main repository under `js/allspeak/LanguagePack_<lang>.js` 2. Published in the `dist/` build for browser use 3. Synced to the Python runtime (`allspeak-py/allspeak/languages/<lang>.json`) 4. Bundled into a starter zip with translated `AGENTS.md` and editor interface 5. Listed on the AllSpeak website as an available language

Estimated time for a single motivated individual with AI assistance to produce a validated language pack: **3–8 weeks**, depending on language complexity and reviewer availability.

---

7. Educational and Community Applications

7.1 The Codex: computational literacy in days

A consistent finding in programming education is that the barrier to entry is not intellectual but temporal and psychological. Learners who cannot experience meaningful progress within hours or days rarely continue. The Codex is designed around this reality: twenty steps, each producing a visible result, each building on the last, with no assumed prior knowledge.

The four-step prompt series for building a simple graphical application extends this: a learner who has completed the Codex can, within a single session, produce something they can show to others. This is not trivial. The experience of producing something real is a significant determinant of whether learning continues.

The Codex is available in all four current languages (EN, FR, DE, IT), with identical pedagogical structure and language-appropriate explanations in each. A learner in French does not receive a translation of the English tutorial; they receive a tutorial written in French, using French keywords, with culturally appropriate examples.

7.2 AllSpeak in practice: the Doclets application

AllSpeak is not only an educational tool. It is also the runtime for a small but complete client–server application: Doclets, a searchable note/document system for a small team, in which the browser client and the MQTT-connected server are both written in AllSpeak (`https://github.com/easycoder/doclets`). Because the stack is deliberately small, one person can understand and change the entire system—UI, messaging, and backend—without a conventional web framework or a separate client language.

The client (`doclets.as`) renders its screens declaratively from Webson JSON and communicates with the server entirely by MQTT request/reply: no polling, and no hand-written API layer. The server (`docletServer.as`) is a short AllSpeak script that subscribes to a topic and dispatches each incoming action to the appropriate handler:

```
on mqtt message append the mqtt message to MessageQueue
...
if Action is `topics` gosub to GetTopics
else if Action is `query` gosub to DoQuery
else if Action is `view` gosub to GetDoclet
```

The one component that is genuinely heavy—managing, searching, and summarising a growing collection of Markdown notes, with optional local-LLM query support (via Ollama) and per-topic access control—lives in a custom plugin (`as_doclets.py`) that exposes simple AllSpeak commands:

```
doclets topics TopicsList from ReceivedMessage
doclets query ResultList from ReceivedMessage
```

This is the "laser" principle of Section 1.3 in miniature: two lines of script conceal semantic embedding, LLM-based ranking, and a token-based access-control layer. The plugin boundary is explicit and visible—everything above the command line is AllSpeak; everything below it is Python.

Doclets also demonstrates the model's incremental quality. Semantic LLM search and per-topic access control were added as features, not rewrites: the architecture absorbed both without restructuring, and the client's state machine grew a flag rather than a new subsystem. The whole system—browser client, MQTT bridge, server script, and plugin—fits a small-team deployment: token-based identity, a browser-only client, and a single Python process behind a static host.

Because AllSpeak separates language from logic, the same architecture can be re-expressed in French, German, or Italian: the keywords resolve automatically through the language pack, and only the script text and user-visible strings need to be authored in the target language. A French-language Doclets client is planned as a demonstration of this property.

AllSpeak's vocabulary is not limited to small applications either. A second deployed system, the Account application, is a spreadsheet-replacement for a one-person event live-streaming service (`https://github.com/easycoder/stream`): approximately 4,500 lines of AllSpeak across five modules (`account-main.as`, `admin-main.as`, `index-main.as`, `build.as`, `seed.as`), in daily use managing real bookings and financial records. Where Doclets demonstrates the breadth of the model in one small system, Account demonstrates its scale.

Together the two applications are a concrete demonstration that AllSpeak's constrained, readable vocabulary is not a limitation. The language that can express a searchable document system, an MQTT server, a plugin boundary, and a production booking system can express a great deal. The readability is not bought at the cost of power.

7.3 The dev.to articles: public positioning

An accompanying article, "AI Doesn't Need Your Programming Language" (published on `dev.to`), presents AllSpeak's argument to a developer audience. The article positions AllSpeak not as a replacement for mainstream languages but as a tool for the era of AI-generated code, where the primary human skill is review rather than authorship. It has served as the primary public-facing introduction to the project's philosophy.

A second article, "AI writes the code, humans review it — review is the coming skill" (`https://dev.to/gtanyware/ai-writes-the-code-humans-review-it-review-is-the-coming-skill-330i`), develops the same position more explicitly: as AI takes over the writing of code, the human skill that matters becomes review, and a language that reads like a human language is what makes that review possible. It takes the Doclets application of Section 7.2 as its working example—a system produced through deliberately small steps in which AI writes and a human reviews, block by block—and argues that customising it is a matter of recognising and adjusting existing code rather than writing new code from scratch.

7.4 Video demonstration

A short video demonstrates AllSpeak in action, walking through a run of the demo build (the colour grid) with a narrated voice-over. The demonstration is intended for readers who want to see the

language working before reading the technical sections — a fifteen-minute introduction to what AllSpeak looks like when it runs.

Video: AllSpeak - a coding language for AI

7.5 The Learn curriculum: documentation as demonstration

Alongside the interactive Codex, AllSpeak maintains a second learning surface: the Learn curriculum at `https://allspeak.ai/learn`. Where the Codex is built around doing—twenty steps, each producing a visible result—Learn is built around reading. Its two tiers mirror the two questions a newcomer actually asks: the Reference pages answer "what is this thing?" and the Idioms pages answer "how do I do X the AllSpeak way?", each idiom with worked examples and explicit anti-patterns.

The pages are written to be followed by a reader with no programming background; the language's vocabulary is the documentation's vocabulary, so the material needs no secondary syntax to explain. The reader itself is an AllSpeak application: the navigation toolbar and the rendered Markdown pane are produced by AllSpeak code running on the same engine the curriculum describes. The documentation is thus also a working demonstration of the platform, and the two learning modes—reading and doing—reinforce one another. The curriculum is published in English and in first-draft French, German, and Italian (each labelled as a draft inviting corrections); as the Codex already demonstrates the four-language structure, the Learn translations follow the same pipeline, with community review via Section 6 the path to full quality.

---

8. Open Questions and Limitations

8.1 Single-maintainer risk

AllSpeak's current development depends substantially on a single technical maintainer. This is a genuine vulnerability for any project seeking to position itself as community infrastructure. Mitigation strategies—contributor onboarding, documentation improvement, institutional partnership—are recognised priorities.

8.2 Language pack governance

As the number of language packs grows, governance questions arise: - Who has authority to accept or reject changes to a language pack? - How are disputes between contributors resolved? - What happens to a language pack whose community contributors become inactive?

These questions do not have settled answers and will require governance structures proportionate to the project's scale. A lightweight model based on language pack maintainers (analogous to Debian's package maintainers or GNOME's translation teams) is the current working proposal.

8.3 Sustainability and succession

The long-term future of a project like this does not rest on funding alone. It rests on finding people who see the same problem and want to work on it. The author is an individual developer who has built AllSpeak privately over years and is now seeking the collaboration needed to take it further—or, if necessary, to see it adopted by someone able to carry it forward.

Funding is relevant only insofar as it enables collaboration: covering the costs of validator time, server infrastructure, and—most importantly—making it possible for people in different places and

different language communities to contribute without bearing the full cost themselves. Travel, coordination, and community building all have associated costs that a small grant (€20k–€50k) would meaningfully address.

But the primary objective is not financial. It is to find the people—linguists, educators, developers, community organisers—who recognise the problem of linguistic exclusion in computational literacy and want to help solve it. The project is open source. Its codebase, its documentation, its methodology, and its full construction history (including the AI-assisted build transcripts preserved in the repository) are available for anyone to inspect, adopt, or extend. The author would prefer to remain involved but is prepared for the possibility that stewardship passes to others, so long as the project continues to serve its purpose.

8.4 The AI dependency

The translation methodology described here relies substantially on AI language model capabilities. This is a practical efficiency, but it introduces a dependency that should be acknowledged: the quality of AI-assisted translation varies by language, and is generally poorer for lower-resource languages—precisely those where AllSpeak's inclusion argument is strongest. Human review is not optional; it is the quality control mechanism that makes AI-assisted translation viable.

8.5 Evidence base

The working paper's claims about declining coding meetup attendance, shifting developer skill profiles, and growing code-review gaps are based on observable patterns and the author's professional experience rather than formal published research. A more rigorous evidence base—survey data, published studies, institutional reports—would strengthen the argument and is a recognised priority for future iterations.

---

9. Conclusion: An Invitation, Not a Pitch

AllSpeak represents one response to a set of questions that the computing community is only beginning to take seriously: what does computational literacy mean when AI writes the code? Who needs to understand that code, and in what language? What tools exist to support them?

The project is at an early stage. Four languages are operational; three await validation by native speakers. A real business application is running on the runtime. The methodology for language onboarding is documented and replicable.

But none of this will matter unless it reaches people who can use it. The author is not an organisation with a grants team and a marketing budget. He is an individual who has worked on this idea privately for a long time, leveraging AI to build something he could not have built alone, and is now putting it forward in the hope that others see its value.

The invitation is straightforward. If you are:

- **A linguist** who understands that programming languages are languages too, and that excluding the world's 6.5 billion non-English speakers from computational literacy is a solvable problem;
- **An educator** who has watched learners bounce off the English keyword barrier and wishes there were another way;
- **A developer** who senses that the era of AI-generated code is going to demand new kinds of reading and verification skills, and that the tools we have are not the tools we will need;
- **A funder or institution** committed to digital inclusion and linguistic diversity, who sees in

AllSpeak a concrete, working instrument rather than a proposal;

- **Anyone** who simply thinks the world should not have to learn English to tell a computer what to do—

Then this is your invitation to get involved.

What does “get involved” mean? It could mean reviewing a language pack for a few hours. It could meanspending an afternoon working through the Codex to see if the pedagogical approach holds up. It could mean contributing code, filing issues, starting a conversation in your own language community, or—if the circumstances align—taking stewardship of the project in your own direction.

AllSpeak is open source. Its full construction history—including the AI-assisted build transcripts that show every dead end, refactor, and decision along the way—is preserved in the repository. There is no hidden agenda, no commercial play, no organisation to join. There is only a working tool and an open question: can we make computational literacy available in every language, for everyone who needs it?

If that question resonates, the repository is at `https://github.com/easycoder/allspeak.ai`. Correspondence can be directed to `info@allspeak.ai`. The rest is up to the people who choose to pick this up.

References

Ethnologue. (2025). *What is the most spoken language?* Ethnologue, 27th Edition. SIL International. https://www.ethnologue.com/

Forrester Research. (2024). *The No-Code/Low-Code Market Will Reach $187 Billion By 2030*. Forrester Research, Inc.

GitHub. (2025). *GitHub Copilot: Two years in, two million paid subscribers*. GitHub Blog. https://github.blog/news-insights/product-news/github-copilot-two-years/

NLnet Foundation. (2025). *Request for Proposals: Digital Commons, Open Source, and Internet Technologies*. NLnet Foundation. https://nlnet.nl/

Scratch Foundation. (2024). *Scratch in Education: Localization and International Impact*. MIT Media Lab. https://scratch.mit.edu/

UNESCO. (2023). *International Fund for Cultural Diversity: Project Guidelines*. UNESCO. https://en.unesco.org/creativity/ifcd

UNESCO. (2024). *AI and the Future of Learning: Policy Recommendations*. UNESCO Digital Library. https://unesdoc.unesco.org/

Resnick, M. et al. (2009). *Scratch: Programming for All*. Communications of the ACM, 52(11), 60–67. https://doi.org/10.1145/1592761.1592779

Solomon, C. (1986). *Computer Environments for Children: A Reflection on Theories of Learning and Education*. MIT Press. (Logo language and educational computing.)

Stanford HAI. (2025). *The GPT Surprise: How AI Chat Tools Changed Engagement in a Large Coding Course*. Stanford Institute for Human-Centered AI. https://hai.stanford.edu/

Appendix: Technical Notes

A.1 Complete opcode reference (127 opcodes)

Flow control: `WHILE`, `IF`, `GOSUB`, `RETURN`, `STOP`, `EXIT`, `CONTINUE`, `FORK`, `WAIT`, `EVERY`, `TRY`, `END_TRY`, `ON_ERROR`

Data manipulation: `PUT`, `APPEND`, `CLEAR`, `SET_DEFAULT`, `SET_BOOLEAN`, `SET_VAR_TYPE`, `SET_ARRAY`, `SET_ELEMENTS`, `INCREMENT`, `DECREMENT`, `ADD`, `SUBTRACT`, `MULTIPLY`, `DIVIDE`, `NEGATE`, `SORT`, `SPLIT`, `FILTER`, `CONVERT`, `ENCODE`, `DECODE`, `INDEX`, `POP`, `PUSH`, `REPLACE`, `TOGGLE`, `DELETE`, `TAKE`

DOM interaction: `CREATE_ELEMENT`, `ATTACH_ELEMENT`, `REMOVE_ELEMENT`, `CLEAR_ELEMENT`, `SET_CONTENT`, `SET_CONTENT_VAR`, `SET_STYLE`, `SET_STYLES`, `SET_ATTRIBUTE`, `SET_ATTRIBUTES`, `REMOVE_ATTRIBUTE`, `SET_CLASS`, `SET_SIZE`, `SET_ID`, `SET_ELEMENT_VALUE`, `ENABLE_ELEMENT`, `DISABLE_ELEMENT`, `FOCUS_ELEMENT`, `FOCUS_ELEMENT`, `CLICK_ELEMENT`, `HIGHLIGHT_ELEMENT`, `SCROLL`, `RENDER`, `SET_TITLE`, `SET_BODY_STYLE`, `SET_HEAD_STYLE`, `SET_TEXT`, `GET_FORM`, `GET_OPTION`

Event handling: `ON_CLICK`, `ON_CHANGE`, `ON_DRAG`, `ON_DROP`, `ON_KEY`, `ON_LEAVE`, `ON_PICK`, `ON_RESUME`, `ON_SWIPE`, `ON_CLICK_DOCUMENT`, `ON_WINDOW_RESIZE`, `ON_CLOSE`, `ON_BROWSER_BACK`, `ON_MESSAGE`, `ON_CALLBACK`

HTTP/API: `REST_GET`, `REST_POST`, `REST_PATH`, `SET_PAYLOAD`, `SET_ENCODING`, `PARAM`, `IMPORT`, `UPLOAD_FILE`

JSON: `JSON_PARSE`, `JSON_FORMAT`, `JSON_SET_LIST`, `JSON_SET_VAR`, `JSON_ADD`, `JSON_DELETE`, `JSON_REPLACE`, `JSON_RENAME`, `JSON_SORT`, `JSON_SHUFFLE`, `JSON_SPLIT`

MQTT messaging: `MQTT_CONNECT`, `MQTT_DISCONNECT`, `MQTT_SUBSCRIBE`, `MQTT_SEND`, `MQTT_ON_CONNECT`, `MQTT_ON_MESSAGE`, `MQTT_TOPIC_INIT`

Storage: `GET_STORAGE`, `PUT_STORAGE`, `REMOVE_STORAGE`, `LIST_STORAGE`

History/navigation: `HISTORY_BACK`, `HISTORY_FORWARD`, `HISTORY_PUSH`, `HISTORY_REPLACE`, `HISTORY_SET`, `NAVIGATE`

Debugging: `DEBUG_PROGRAM`, `DEBUG_STEP`, `DEBUG_STOP`, `DEBUG_SYMBOL`, `DEBUG_SYMBOLS`, `TRACE_RUN`, `TRACE_SETUP`, `SET_TRACER_ROWS`

Module system: `DECLARE_MODULE`, `RUN_MODULE`, `CLOSE_MODULE`, `DECLARE_ALIAS`, `DECLARE_CALLBACK`, `DECLARE_SYMBOL`, `DECLARE_VARIABLE`, `DECLARE_ELEMENT`, `IMPORT`

Media: `PLAY_AUDIO`, `FULLSCREEN`

UI elements: `GET_FORM`, `GET_OPTION`, `SET_SELECT`, `SET_DEFAULT`, `SANITIZE`

Utility: `LOG`, `ALERT`, `CONFIRM`, `PRINT`, `MAIL`, `COPY_TO_CLIPBOARD`, `DUMMY`

Other: `SEND_MESSAGE`, `SET_READY`, `SET_PROPERTY`, `SET_ARG`, `SET_PAYLOAD`, `GET_ARG`, `REQUIRE`

A.2 Language pack structure specification

A language pack file is a JavaScript file defining a single global variable `AllSpeak_LanguagePack_<code>` containing an object with the following top-level keys:

```
{
  "meta": {
    "language": "fr",
    "label": "Français",
    "version": "0.1.0"
  },
  "opcodes": {
    "ADD": { "keyword": "ajouter", "patterns": ["ajouter {valeur} à {variable}", ".
    ...
  },
  "connectors": { "to": "à", "into": "dans", ... },
  "literals": { "true": "vrai", "false": "faux", ... },
  "timeUnits": { "second": "seconde|secondes", ... },
  "conditions": { "is": "est", "greater": "plus grand|supérieur", ... },
  "diagnostics": {
    "undeclaredVariable": "Variable non déclarée '{name}'",
    ...
  },
  "words": {
    "giving": "donnant",
    "add": "ajouter",
    ...
  }
}
```

The `words` section maps every canonical term to one or more surface forms, separated by `|`. Multiple variants support gender agreement, verb conjugation, and orthographic variation (e.g., accented/unaccented forms).

A.3 Cross-referencing: mapping opcodes to source files

Opcode domain	Handler file	Examples
Core operations	`js/allspeak/Core.js`	ADD, PUT, WHILE, IF, GOSUB
DOM/browser	`js/allspeak/Browser.js`	CREATE_ELEMENT, ON_CLICK, SET_STYLE
HTTP/REST	`js/allspeak/REST.js`	REST_GET, REST_POST, REST_PATH
JSON manipulation	`js/allspeak/JSON.js`	JSON_PARSE, JSON_SORT, JSON_ADD
MQTT messaging	`js/allspeak/MQTT.js`	MQTT_CONNECT, MQTT_SEND
Conditions	`js/allspeak/Condition.js`	Condition resolution for IF/WHILE
Values	`js/allspeak/Value.js`	Value resolution and type conversion
Runtime execution	`js/allspeak/Run.js`	Opcode execution, variable management
Compilation	`js/allspeak/Compile.js`	Tokenisation, language directive, handler dispatch
Language layer	`js/allspeak/Language.js`	Word resolution, diagnostics lookup

A.4 Weblate workflow

1. A new language is requested or proposed via the project repository
2. A component is created in the Weblate instance for the language
3. The reference English pack is loaded as the source
4. AI-assisted initial translation is imported as the starting draft
5. Community reviewers propose, discuss, and approve changes
6. When the translation reaches the defined quality threshold, a pull request is created against the main repository
7. The language pack is merged, built, and released

This working paper is a living document. The latest version is maintained at `docs/whitepaper.md` *in the AllSpeak repository. For citation purposes, please reference the versioned PDF available via the project's Codeberg repository.*